

UART Communication and Data Integrity

Module	ET4047
Date	04/10/2026
Lab No.	Lab 5
Author	Luke Griffin

1 Setup

Equipment

Item	Qty	Notes
Arduino Nano (classic) or Nano Every, on Uno-style breakout shield	2	All the code works on either board. Task 1 detects the board automatically.
16x2 LCD with I2C backpack, or Adafruit RGB LCD shield	1-2	Either works: there is a folder of code for each display.
Jumper wires (male-male / male-female)	~10	
USB cables	2	One per board.
Push button	1	Task 4 only.

Software

- Arduino IDE. Select the board you have: *Tools* → *Board* → *Arduino AVR Boards* → *Arduino Nano* (classic), or *Arduino megaAVR Boards* → *Arduino Nano Every* (install the megaAVR package from the Boards Manager first). Classic Nano clones may also need *Tools* → *Processor* → *ATmega328P (Old Bootloader)*.
- **The library for your display only:** LiquidCrystal I2C (by Frank de Brabander) for the I2C backpack LCD, or Adafruit RGB LCD Shield Library (by Adafruit) for the RGB shield. *Tools* → *Manage Libraries...* and search for it.
- `SoftwareSerial` is built in. There is nothing to install.
- **The lab code, `W05_UART_Lab_Code.zip`.** Unzip it somewhere you can find it again.

Choose the code for your display

The zip has one folder for each type of display. Use **only** the folder for your display. Every sketch is a single .ino file that opens with *File* → *Open*, and the comments in it explain each step.

Display	How to recognise it	Folder to use
16x2 LCD with I2C backpack	A separate screen with a small board on the back and four wires	<code>I2C_Backpack_LCD</code>
Adafruit RGB LCD shield	Plugs straight onto the breakout board and has five buttons	<code>Adafruit_RGB_LCD_Shield</code>

LCD wiring (all tasks)

I2C LCD connections

From		To	Purpose
LCD GND	→	Nano GND	Ground
LCD VCC	→	Nano 5V	Power
LCD SDA	→	Nano A4	I2C data
LCD SCL	→	Nano A5	I2C clock

LCD wiring (every task)

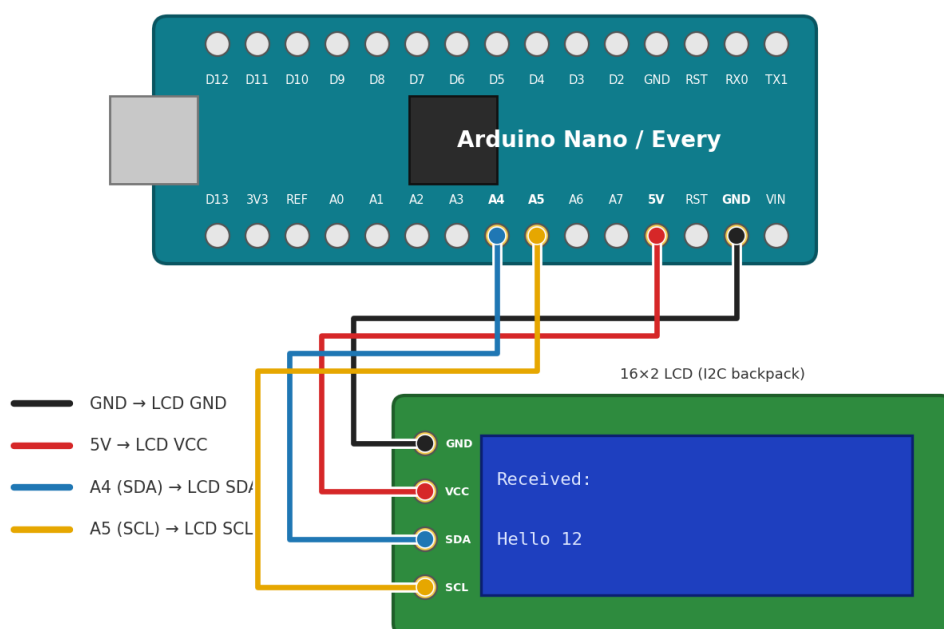


Figure 1: LCD wiring. The pinout is the same on the classic Nano and the Nano Every.

Using the Adafruit RGB LCD shield instead? Plug it onto the Uno-style breakout shield. It uses the same I2C pins (SDA on A4, SCL on A5), so no wiring is needed.

2 Task 1: UART loopback on a single board

Objective: connect the board's UART output straight back into its own input. Everything the board transmits should arrive back on its receive pin, which proves that the UART, the wiring and your code are working before you involve a second board.

Wiring

Loopback connection (plus the LCD wiring from Section 1)

From		To	Purpose
Nano D1 (TX)	→	Nano D0 (RX)	Loopback: output feeds straight into input

Task 1: UART loopback

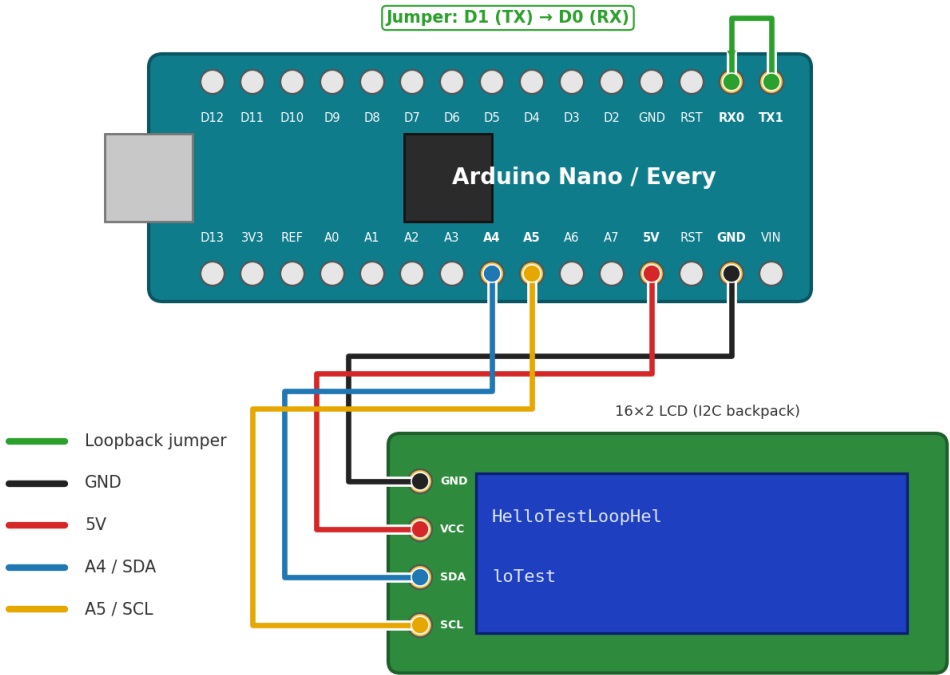


Figure 2: Task 1 wiring: one jumper from D1 (TX) to D0 (RX), which are next to each other on the header

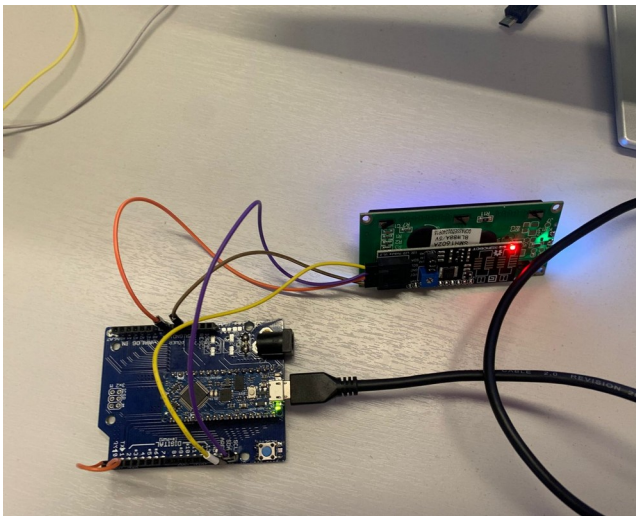


Figure 3: Loopback jumper and LCD wiring



Figure 4: LCD without line wrapping

	Classic Nano (ATmega328P)	Nano Every (ATmega4809)
Hardware UARTs on the pins	One, <code>Serial</code> . It is shared with the USB link.	A spare one, <code>Serial1</code> . USB has its own channel.
Port used by the Task 1 code	<code>Serial</code>	<code>Serial1</code>
Loopback jumper while uploading	Remove it , or the upload fails	Can stay in place
Serial Monitor during Task 1	Shows the messages being sent. Don't type in it : typed characters also arrive on D0.	Stays empty

Why the classic Nano behaves differently

On the classic Nano, D0 and D1 are also wired to the USB-to-serial chip. Anything the board sends on D1 goes both to the jumper and to your PC, and the upload uses the same pins, so the jumper gets in the way while uploading.

What happens

Code

Upload `Task1_Loopback` from your display's folder. On a classic Nano, take the jumper off while uploading.

Once a second the board sends a word (Hello, Test or Loop) out of its TX pin, D1. The jumper carries it straight back into its RX pin, D0, and every character that arrives is shown on the LCD. When a row is full the text moves down to the next row; when both rows are full the screen clears and starts again at the top.

Expected result: the display fills row by row with `HelloTestLoopHel / loTestLoopHelloT`, clears, and starts again at the top.

Task 1 questions

1a Run it

Upload the code, wire the jumper and check the messages appear. **(record)** Take a photo of the LCD.

1b Break the loop

While the sketch is running, pull the D1-D0 jumper out, wait a few seconds, then put it back. What happens on the LCD, and why? Is anything lost while the wire is out?

Your answer:

1c Without the wrapping

In the receive loop, temporarily replace `lcdPutChar(c);` with `lcd.write(c);` and run it again. Describe what you see (compare with Figure 4), then put `lcdPutChar()` back. Why does the LCD not wrap on its own?

Your answer:

1d Timing

Each character is sent as a 10-bit frame (1 start bit, 8 data bits, 1 stop bit), and at 9600 baud each bit lasts 1/9600 s. How long does it take to transmit the word `Hello` at 9600 baud? How many characters per second can a 9600-baud link carry at most?

Your answer:

3 Task 2: Communication between two boards

Objective: Board A (the sender) transmits a numbered message every second; Board B (the receiver) collects the characters into a complete line and shows it on its LCD and Serial Monitor.

Wiring

Label your boards **A** (sender) and **B** (receiver). The LCD connects to **Board B** only.

Board-to-board connections (the TX/RX wires cross over)

From		To	Purpose
A D11 (TX)	→	B D10 (RX)	A's data travels to B
A D10 (RX)	→	B D11 (TX)	Return path (not used this week)
A GND	→	B GND	Common reference voltage. Essential.

Task 2: two boards, TX/RX crossed over

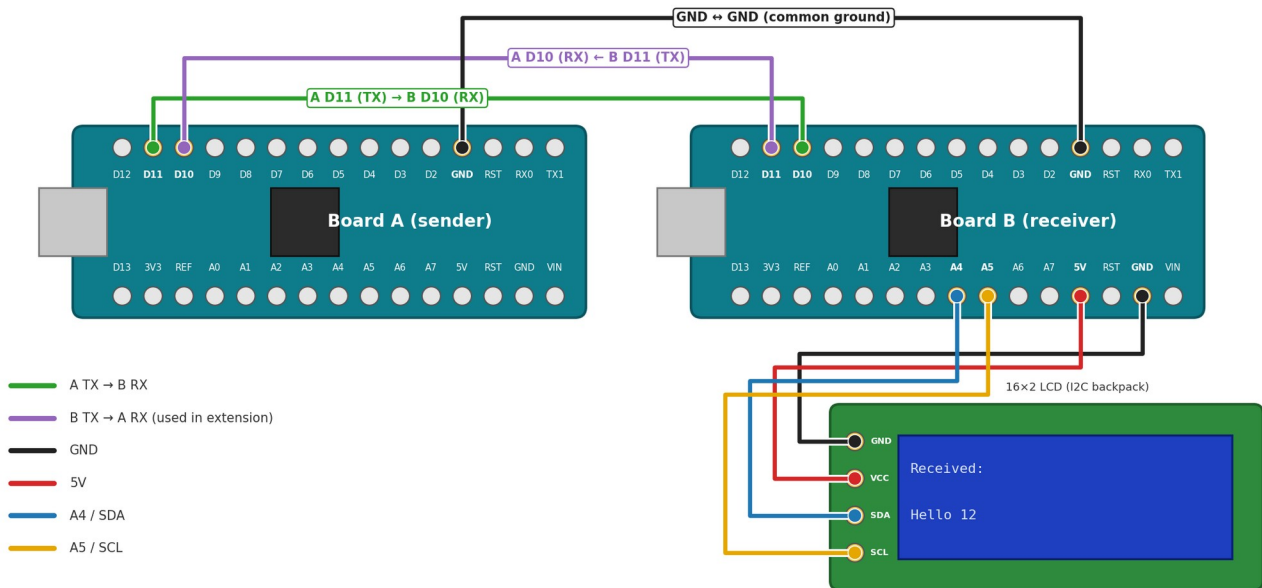


Figure 5: Task 2 wiring: TX goes to RX in both directions, grounds joined, LCD on Board B

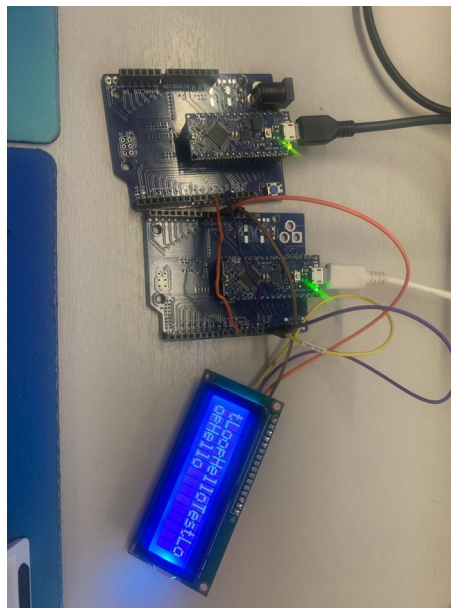


Figure 6: The two-board set-up in the lab

Connect both boards to the PC by USB. Each gets its own COM port. Upload the sender code to A and the receiver code to B, selecting the correct port each time. Open a Serial Monitor for each board, both at **9600 baud**. (In the IDE, open a second window with *File → New* to get a second monitor.)

What happens

Code

Upload `Task2_Sender` to Board A and `Task2_Receiver` to Board B, both from your display's folder.

Every second Board A sends a numbered message, such as Hello 12, with a newline character on the end. UART only delivers one character at a time, so Board B saves each character in a buffer (an array) until the newline arrives. Then it knows the message is complete and shows it on its LCD and Serial Monitor.

Task 2 questions

2a Run it

Upload both sketches and run them. **(record)** Paste a few lines from **both** Serial Monitors. They should look like this:

Board A Serial Monitor

```
Sender ready
Sent: Hello 0
Sent: Test 1
Sent: Loop 2
```

Board B Serial Monitor

```
Received: Hello 0
Received: Test 1
Received: Loop 2
```

2b Break it on purpose

Make each change below **one at a time**, observe what Board B shows, then undo it. **(record)** Fill in the table and explain each result. Think about the baud rate (both boards must use the same one), which pin drives which (TX must go to RX), and the shared ground.

Change	What Board B shows	Why
Remove the GND-GND wire		
Change the receiver to <code>link.begin(4800)</code>		
Connect A's TX to B's TX instead of RX		
Set <code>MAX_LEN</code> to 4 on the receiver		

Note

A missing ground sometimes *appears* to work, because the boards may share a ground through the PC's USB ports. It is still wrong, and it fails as soon as one board runs from a battery.

2c Questions

1. Why does the receiver need the `'\n'` at the end of each frame? What would B display if the sender left it out?
2. What would happen if the `len < MAX_LEN` check were removed and A sent a 100-character message?

Your answer:

4 Task 3: Secret messages with a Caesar cipher

Objective: anyone who connects to the wire can read what Board A sends. In this task Board A scrambles (encrypts) each message with a Caesar cipher before sending it, and Board B unscrambles (decrypts) it. The wiring is the same as Task 2.

The Caesar cipher

Every letter is replaced by the letter a fixed number of places further along the alphabet. That number is the **key**, also called the shift. With a shift of 3, A becomes D, B becomes E, and so on. At the end the alphabet wraps round: X becomes A, Y becomes B and Z becomes C. To decrypt, move each letter back by the same number. Spaces and digits are not changed.

Message (plain text)	Sent on the wire (shift 3)
Hello	Khoor
Embedded	Hpehgghg
Software	Vriwzduh

What happens

Code

Upload `Task3_Sender` to Board A and `Task3_Receiver` to Board B, both from your display's folder.

Every 2 seconds Board A encrypts the next message with a shift of 3 and sends the scrambled text. Board B shows the scrambled text it received on the top row of the LCD and the decrypted message on the bottom row. Both Serial Monitors show the message before and after encryption.

Task 3 questions

3a Run it

Upload both sketches and run them. **(record)** Paste a few lines from both Serial Monitors. They should look like this:

Board A Serial Monitor

```
Sender ready
Plain: Hello    Sent: Koor
Plain: Embedded Sent: Hpehgghg
```

Board B Serial Monitor

```
Received: Koor    Decrypted: Hello
Received: Hpehgghg Decrypted: Embedded
```

3b The wrong key

Change **SHIFT** to 4 in the **receiver only** and upload it again. What does Board B show for Hello? Why must both boards use the same key? Put it back to 3 afterwards.

Your answer:

3c By hand

1. Encrypt the word Zebra with a shift of 3. Which letter has to wrap round to the start of the alphabet?
2. Decrypt Pdwkv (shift 3).

Your answer:

3d Is it secure?

1. How many different keys can a Caesar cipher have?
2. Someone listening on the wire sees Wkh grru frgh lv 1234. How could they read it without knowing the key?
3. Why is a Caesar cipher not used to protect real data?

Your answer:

5 Task 4: Checking messages with SHA-1

Objective: noise or a loose wire can change bits on the way, and UART does not notice. In this task Board A sends a **hash** with every message, and Board B uses it to check whether the message arrived unchanged. The wiring is the same as Task 2, plus a push button on Board A that damages a message on purpose.

Task 3: two boards + tamper button on Board A

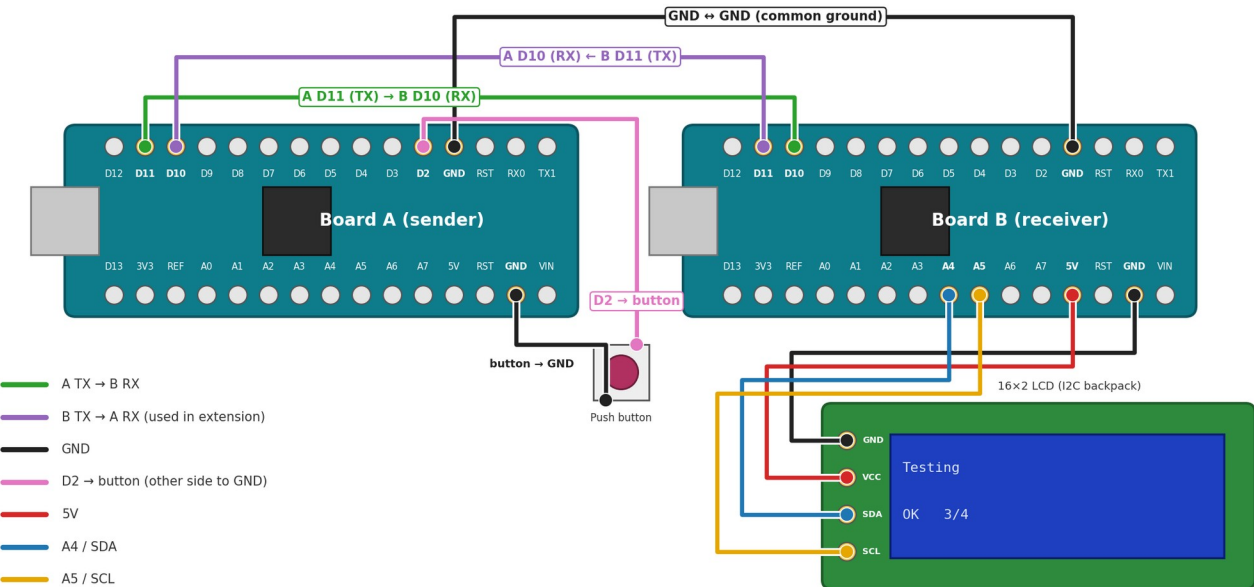


Figure 7: Task 4 wiring: the Task 2 link, plus a push button between Board A D2 and GND

What is a hash?

A hash function turns any message into a short, fixed-size “fingerprint”. SHA-1 always gives 40 hexadecimal characters. The same message always gives the same hash, but changing even one letter gives a completely different one:

Message	SHA-1 hash
Testing	0820b32b206b7352858e8903a838ed14319acdfd
Tasting	ea1e43d415e84b553d0625a9ac75af4ea2a27fe9

So if Board B works out the hash of the message it received and gets the same answer as Board A, the message is almost certainly unchanged. If the two hashes are different, something was damaged on the way.

What Board A sends

Testing		0820b32b...319acdfd	\n
message (any length)	separator	SHA-1 hash: 40 hex characters	end of frame

What happens

Code

Upload **Task4_Sender** to Board A and **Task4_Receiver** to Board B, both from your display's folder. The SHA-1 code is already at the bottom of both sketches: you do not need to change it.

Every 2 seconds Board A works out the hash of the next message and sends the message and the hash together. Board B works out the hash of the message it received and compares the two. The LCD shows the message on the top row, and OK or FAIL with a running count (e.g. OK 3/4) on the bottom row. While you hold the button on Board A, it changes one bit of the message before sending it, but still sends the hash of the original, just like a message damaged on the wire.

Task 4 questions

4a Run it

Upload both sketches and run them. **(record)** Paste one result from Board B's Serial Monitor. Every message should pass:

Board B Serial Monitor

```
Message: Testing
Received: 0820b32b206b7352858e8903a838ed14319acdfd
Computed: 0820b32b206b7352858e8903a838ed14319acdfd
Result: OK
```

4b Damage a message

Hold the button on Board A while a message is sent. **(record)** Paste the FAIL result from Board B and the matching line from Board A. They should look like this:

Board A Serial Monitor (button held)

```
Sent (TAMPERED): Dmbedded|0fd17b94c9576216ee0a47001e3fdbb1920a1f50
```

Board B Serial Monitor

```
Message: Dmbedded
Received: 0fd17b94c9576216ee0a47001e3fdbb1920a1f50
Computed: 106b572c38133fff8835b5165f4d092b9042db0e
Result: FAIL
```

Only one bit of one letter changed (E became D). Roughly how many of the 40 hash characters are different?

Your answer:

4c Questions

1. Can Board B fix a damaged message using the hash, or only tell that it is damaged?
2. If the damage hits the hash instead of the message, what does Board B report? Is that the right result?
3. Testing is 7 characters, but the line sent is 49 characters long. Why might a smaller check be better on a slow link?

Your answer: